$$\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial \hat{y}}\left[\frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}} + \frac{\partial \hat{y}}{\partial o_{22}} \times \frac{\partial o_{22}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}}\right]$$

↳ Very complex with only 2 hidden layers

Backpropagatⁿ → chain diff (rule) + Memoization

↓ Maths ↓ CS (OP)

↳ Store the derivatives while calculated one time.

Vanishing Gradient Problem

less in fewer layer networks ,VGD is affected mostly in deep layer networks

In ML, the vanishing gradient problem is encountered when training ANNs with GD and backpropagation. In such methods, during each iteration of train each of the NN's weights receives an update proportional to the partial derivative of the error function w.r.t. the current weight. In worst case, this may stop the training of NN.

gradient
layer4->0.5
layer 3->0.05
layer2->0.05
layer 1->0.0005

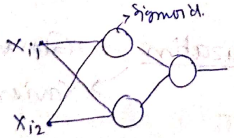
1) $0.1 \times 0.1 \times 0.1 \times 0.1 = \boxed{0.0001}$ → VGP.

2) Deep NN →

early layers affected more as gradient needs to travel further during backpropagation

3) Sigmoid/tanh → Activatⁿ fⁿ →

Back prop.



$\hat{y} - y = L$

$w_n = w_0 - \eta \boxed{\frac{\partial L}{\partial w_0}}$ → derivative of the L wrt w

$$\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}}$$

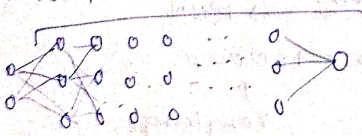
↑ $0<x<1$ ↑ $0<x<1$

$w_n = 1 - 0.01 \times 0.0001$ (η)

$w_n = 0.99999$

did not changed

0.0001



too many layers.

$\frac{\partial L}{\partial w'_{11}} = 0.000000001$

↳ stop training.

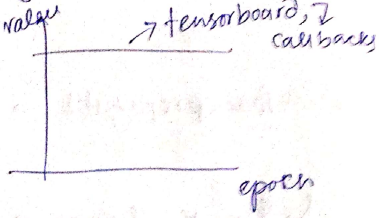
How to recognize VGP?

1) Loss focus → epoch → no changes → VGP

2) weights → graph

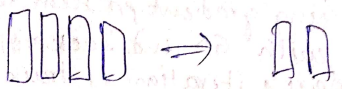
3) Gradient ->graph

keras → loss after every epoch



How to handle Vanishing Gradient Problem?

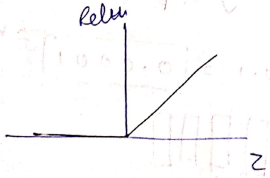
1) Reduce model complexity



→ problem cannot find complex patterns

2) Using ReLU Activation functions

max (0, z)



$\boxed{0}, \boxed{1}$

$= 1 \times 1 \times 1 = 1$ → VGP

dying relu → derivative $w_n = w_0$

{leaky relu}

3) Proper weight initialization → Glorat, Xavier.

4) Batch normalization → Layer. pg 25

normalizing ii/p of each layer with constant mean and variance

5) Residual Network → RNN (ResNET)

↳ Building Block → ANN

(Skips connection) Bypassing one or more layers

Exploding Gradient Problems. → RNN

$10 \times 10 \times 10 \ldots \times 10 = 100000\ldots$

↑ >1 ↑ >1 ↑ >1 ↑ very large.

Opposite of VGD

layer 4->2.0
layer 3->4.0
layer 2->8.0
layer 1->16.0

Solution
-Gradient clippinh
-lower learning rate
-weight regularization(l1,L2)
-Gradient Normalization

Scanned with OKEN Scanner
Scanned with OKEN Scanner